UPF

INSTITUTO DE TECNOLOGIA - ITec

Algoritmos e Programação

Prof. Eder Pazinatto

Modularização – Funções (parte II)

Funções - Tipos

- As funções podem ser classificadas em integradas (built-in functions), funções definidas pelo usuário e anônimas (ou funções lambda).
  - **1.Funções Integradas**: Python fornece várias funções integradas como print(), input(), len(), etc.
  - **2.Funções Definidas pelo Usuário**: funções que os desenvolvedores criam para realizar uma tarefa específica, como mostrar uma mensagem.
  - **3.Funções Lambda**: função anônima. Uma função lambda pode receber qualquer número de argumentos, mas só pode ter uma expressão.
  - **4.Funções recursivas**: função é dita recursiva se chama a si mesma.
  - **5. Funções de ordem superior**: função que aceita outras funções como parâmetros, ou retorna uma função como resultado.

Funções - Tipos

- **1. Funções Integradas**: também chamadas de built-in functions, são funções que já fazem parte da linguagem e podem ser utilizadas diretamente no programa, sem a necessidade de defini-las com ***def*** ou importar bibliotecas
- A manipulação de strings em Python é feita de forma prática usando métodos *embutidos*
- Alguns exemplos:

  Concatenação (+): Junta duas strings. Exemplo: "Olá, " + "Ana" resulta em "Olá, Ana".

  Repetição (*): Repete a string várias vezes. Exemplo: "-" * 5 resulta em "-----".

  Fatiamento (*Slicing* [início:fim:passo]): Acessa partes da string. Os índices começam em 0, e -1 pega o último caractere. Exemplo: "Python"[0:2] resulta em "Py".

- Fonte de pesquisa: https://www.w3schools.com/python/python_strings.asp

Funções - Tipos

- **2. Funções Definidas pelo Usuário**: o usuário define o nome e a funcionalidade da função

```
def analisar_texto(texto):
    print("Texto original:", texto)
    print("Texto em maiúsculas:", texto.upper())
    print("Quantidade de caracteres:", len(texto))


# Programa principal
frase = input("Digite uma frase: ")

analisar_texto(frase)
```

Funções - Tipos

- **3. Funções Lambda**: função lambda é uma pequena função anônima
- Função para somar 2 números

```
soma = lambda a, b: a + b

print(soma(10, 5))
```

- Co

Função tradicional	Função lambda
`def dobro(x):`	`dobro = lambda x:`
`return x * 2`	`x * 2`
Pode conter várias instruções	Contém uma única expressão

Funções - Tipos

- **4. Funções recursivas**: chama a si mesma

- Utilizamos uma função recursiva em Python quando uma função precisa chamar a si mesma para resolver um problema, dividindo-o em problemas menores até chegar a uma condição de parada.

- A recursividade é bastante utilizada em cálculos matemáticos, percursos em árvores, busca de arquivos em diretórios e resolução de problemas que podem ser divididos em etapas semelhantes.

Funções - Tipos

- **4. Funções recursivas**: chama a si mesma

- Exemplo:

```
def soma(numero):
    if numero == 0:
        return 0
    else:
        return numero + soma(numero-1)

print(soma(5))  # Saida: 15
```

A função **soma** recebe um número.
Se número é 0, a função retorna 0.

Caso contrário, a função retorna o número mais a função **soma** no número (número -1).

A recursão continua até que a condição base seja atingida (ou seja, até que número seja 0).

Soma(5) = Retorna 5 + 4 + 3 +2 +1 + 0 SAIDA = 15

Funções - Tipos

- **5. Funções de ordem superior**: função que aceita outras funções como parâmetros ou retornando-as como resultado

- Função para somar 2 números

```
def calcular(a, b, operacao):
    return operacao(a, b)

soma = lambda x, y: x + y
multiplicacao = lambda x, y: x * y

print(calcular(5, 3, soma))
print(calcular(5, 3, multiplicacao))
```

Funcionamento?
A função **calcular()** recebe três parâmetros: a, b e operacao.

O parâmetro operacao recebe uma função (soma ou multiplicacao).

Dentro de calcular(), a expressão operacao(a, b) executa a função recebida, utilizando os valores 5 e 3.